

```
In [54]: # PARTE 0.0: IMPORTAR LIBRERÍAS Y DEFINIR RUTA DE DATOS

# Librerías para manejar datos y archivos
import pandas as pd
import numpy as np
import os
import glob

# Librerías para gráficos
import matplotlib.pyplot as plt
import seaborn as sns

# Configuración para ver más columnas en las tablas
pd.set_option("display.max_columns", 100)

# Definimos la carpeta donde están los archivos CSV
# Esta es tu ruta en Mac. Si cambiaste el nombre de la carpeta, ajústala aquí
DATA_DIR = "/Users/robertoverdejo/Desktop/tarea 3 datos"

# Buscamos automáticamente los archivos de cada tipo
census_files = glob.glob(os.path.join(DATA_DIR, "census_team*.csv"))
eno_files = glob.glob(os.path.join(DATA_DIR, "eno_team*.csv"))
grd_files = glob.glob(os.path.join(DATA_DIR, "grd_team*.csv"))

# Mostramos cuántos archivos encontró Python
print("Archivos Census encontrados:", len(census_files))
print("Archivos ENO encontrados:", len(eno_files))
print("Archivos GRD encontrados:", len(grd_files))

# Mostramos los nombres de los archivos encontrados
print("\nArchivos Census:")
for file in census_files:
    print(os.path.basename(file))

print("\nArchivos ENO:")
for file in eno_files:
    print(os.path.basename(file))

print("\nArchivos GRD:")
for file in grd_files:
    print(os.path.basename(file))
```

Archivos Census encontrados: 15

Archivos EN0 encontrados: 15

Archivos GRD encontrados: 15

Archivos Census:

census_team16.csv
census_team17.csv
census_team03.csv
census_team15.csv
census_team01.csv
census_team21 (1).csv
census_team05.csv
census_team11.csv
census_team07.csv
census_team13.csv
census_team4_Santiago.csv
census_team06.csv
census_team08.csv
census_team09.csv
census_team18.csv

Archivos EN0:

eno_team11.csv
eno_team05.csv
eno_team06.csv
eno_team13.csv
eno_team07.csv
eno_team03.csv
eno_team17.csv
eno_team16.csv
eno_team01.csv
eno_team15.csv
eno_team4_Santiago.csv
eno_team18.csv
eno_team09.csv
eno_team21.csv
eno_team08.csv

Archivos GRD:

grd_team4_Santiago.csv
grd_team13.csv
grd_team07.csv
grd_team06.csv
grd_team11.csv
grd_team05.csv
grd_team01.csv
grd_team15.csv
grd_team16.csv
grd_team03.csv
grd_team17.csv
grd_team21 (1).csv
grd_team18.csv
grd_team08.csv
grd_team09.csv

In [55]: *# PARTE 0.1: LECTURA Y CONCATENACIÓN DE ARCHIVOS*

```
# Leemos y juntamos todos los archivos Census en una sola tabla
census_master = pd.concat(
    [pd.read_csv(file) for file in census_files],
    ignore_index=True
)

# Leemos y juntamos todos los archivos ENO en una sola tabla
eno_master = pd.concat(
    [pd.read_csv(file) for file in eno_files],
    ignore_index=True
)

# Leemos y juntamos todos los archivos GRD en una sola tabla
grd_master = pd.concat(
    [pd.read_csv(file) for file in grd_files],
    ignore_index=True
)

# Mostramos el tamaño de cada tabla
# El primer número son filas y el segundo columnas
print("Tamaño census_master:", census_master.shape)
print("Tamaño eno_master:", eno_master.shape)
print("Tamaño grd_master:", grd_master.shape)

# Mostramos las primeras filas para revisar que se cargó bien
display(census_master.head())
display(eno_master.head())
display(grd_master.head())
```

Tamaño census_master: (37, 13)

Tamaño eno_master: (37, 8)

Tamaño grd_master: (37, 13)

	codigo_comuna	nombre_comuna	pop_total	pop_chilean	pop_foreign	pct_foreign
0	13122	Peñalolén	236478	216664	18304	7.740255
1	13605	Peñaflor	94402	90616	3235	3.426834
2	13114	LAS CONDES	292739	261670	31069	10.613208
3	13503	CURACAVÍ	34977	33517	1460	4.174172
4	13109	La Reina	103157	82209	19762	19.380000

	codigo_comuna	nombre_comuna	eno_total	eno_chilean	eno_foreign	eno_desconoc
0	13102	Cerrillos	642	297	101	
1	13105	Estación Central	1212	634	111	
2	13302	Lampa	672	390	93	
3	13120	Ñuñoa	1735	830	255	
4	13129	San Joaquín	767	372	161	

	codigo_comuna	nombre_comuna	grd_total	grd_chilean	grd_foreign	grd_pct_foreign
0	13101	Santiago	54929	35664	19265	35.07254
1	13502	Alhué	900	793	107	11.89000
2	13602	El Monte	4114	4015	99	2.41000
3	13107	Huechuraba	6170	5743	427	6.92000
4	13112	La Pintana	25556	24778	778	3.04000

```
In [56]: # Creamos una función para revisar rápidamente cada tabla
def revisar_tabla(df, nombre):
    # Imprime información básica de la tabla
    print(f"---- {nombre} ----")

    # Cantidad total de filas
    print("Filas:", len(df))

    # Cantidad de comunas distintas según codigo_comuna
    print("Comunas únicas:", df["codigo_comuna"].nunique())
```

```

# Cantidad de códigos de comuna duplicados
print("Duplicados en codigo_comuna:", df["codigo_comuna"].duplicated().sum())

# Mostramos algunas comunas para revisar visualmente
print("Primeras comunas:", df["nombre_comuna"].head(10).tolist())
print()

# Aplicamos la revisión a las tres tablas
revisar_tabla(census_master, "Census")
revisar_tabla(eno_master, "ENO")
revisar_tabla(grd_master, "GRD")

# Convertimos los códigos de comuna de cada tabla en conjuntos
# Esto permite comparar qué comunas están o faltan entre bases
census_codes = set(census_master["codigo_comuna"])
eno_codes = set(eno_master["codigo_comuna"])
grd_codes = set(grd_master["codigo_comuna"])

# Revisamos diferencias entre bases
print("Comunas en Census pero no en ENO:", census_codes - eno_codes)
print("Comunas en Census pero no en GRD:", census_codes - grd_codes)
print("Comunas en ENO pero no en Census:", eno_codes - census_codes)
print("Comunas en GRD pero no en Census:", grd_codes - census_codes)

```

--- Census ---

Filas: 37

Comunas únicas: 36

Duplicados en codigo_comuna: 1

Primeras comunas: ['Peñalolén', 'Peñaflor', 'LAS CONDES', 'CURACAVÍ', 'La Reina', 'Quilicura', 'Tiltil', 'Providencia', 'San Miguel', 'Paine']

--- ENO ---

Filas: 37

Comunas únicas: 36

Duplicados en codigo_comuna: 1

Primeras comunas: ['Cerrillos', 'Estación Central', 'Lampa', 'Ñuñoa', 'San Joaquín', 'San José de Maipo', 'San Bernardo', 'Calera de Tango', 'La Pintana', 'Huechuraba']

--- GRD ---

Filas: 37

Comunas únicas: 36

Duplicados en codigo_comuna: 1

Primeras comunas: ['Santiago', 'Alhué', 'El Monte', 'Huechuraba', 'La Pintana', 'María Pinto', 'Lo Prado', 'Pudahuel', 'Calera de Tango', 'San Bernardo']

Comunas en Census pero no en ENO: set()

Comunas en Census pero no en GRD: set()

Comunas en ENO pero no en Census: set()

Comunas en GRD pero no en Census: set()

```

In [57]: # Identificamos qué comunas están duplicadas en cada tabla
# keep=False muestra todas las filas que tienen un codigo_comuna repetido

print("Duplicados en Census:")

```

```

display(
  census_master[census_master["codigo_comuna"].duplicated(keep=False)]
  .sort_values("codigo_comuna")
)

print("Duplicados en ENO:")
display(
  eno_master[eno_master["codigo_comuna"].duplicated(keep=False)]
  .sort_values("codigo_comuna")
)

print("Duplicados en GRD:")
display(
  grd_master[grd_master["codigo_comuna"].duplicated(keep=False)]
  .sort_values("codigo_comuna")
)

```

Duplicados en Census:

	codigo_comuna	nombre_comuna	pop_total	pop_chilean	pop_foreign	pct_foreign
0	13122	Peñalolén	236478	216664	18304	7.740255
5	13122	Quilicura	236478	214770	20141	8.570000

Duplicados en ENO:

	codigo_comuna	nombre_comuna	eno_total	eno_chilean	eno_foreign	eno_descon
16	13122	Quilicura	1487	701	124	
20	13122	Peñalolén	1487	701	124	

Duplicados en GRD:

	codigo_comuna	nombre_comuna	grd_total	grd_chilean	grd_foreign	grd_pct_fore
22	13122	Peñalolén	44917	42047	2870	6
25	13122	Quilicura	13062	11120	1942	14

In [58]: *# Corregimos manualmente el código de comuna de Quilicura*
El problema detectado es que Quilicura aparece con codigo_comuna 13122,
que también está siendo usado por Peñalolén.
Para evitar duplicados, asignamos a Quilicura su código comunal correcto:

```

census_master.loc[
    census_master["nombre_comuna"].str.upper().str.contains("QUILICURA", na=
        "codigo_comuna"
    ] = 13125

eno_master.loc[
    eno_master["nombre_comuna"].str.upper().str.contains("QUILICURA", na=Fa
        "codigo_comuna"
    ] = 13125

grd_master.loc[
    grd_master["nombre_comuna"].str.upper().str.contains("QUILICURA", na=Fa
        "codigo_comuna"
    ] = 13125

# Revisamos nuevamente si quedan duplicados en cada tabla
print("Duplicados Census:", census_master["codigo_comuna"].duplicated().sum()
print("Duplicados ENO:", eno_master["codigo_comuna"].duplicated().sum())
print("Duplicados GRD:", grd_master["codigo_comuna"].duplicated().sum())

# Revisamos que Peñalolén y Quilicura ahora tengan códigos distintos
display(census_master[census_master["nombre_comuna"].str.upper().str.contain
display(eno_master[eno_master["nombre_comuna"].str.upper().str.contains("PEÑ
display(grd_master[grd_master["nombre_comuna"].str.upper().str.contains("PEÑ

```

Duplicados Census: 1

Duplicados ENO: 1

Duplicados GRD: 1

	codigo_comuna	nombre_comuna	pop_total	pop_chilean	pop_foreign	pct_foreign
0	13122	Peñalolén	236478	216664	18304	7.740255
5	13125	Quilicura	236478	214770	20141	8.570000

	codigo_comuna	nombre_comuna	eno_total	eno_chilean	eno_foreign	eno_descon
16	13125	Quilicura	1487	701	124	
20	13122	Peñalolén	1487	701	124	

	codigo_comuna	nombre_comuna	grd_total	grd_chilean	grd_foreign	grd_pct_fore
22	13122	Peñalolén	44917	42047	2870	6
25	13125	Quilicura	13062	11120	1942	14

In [59]: *# Revisamos nuevamente qué comunas siguen duplicadas después de la corrección*

```
print("Duplicados actuales en Census:")
display(
    census_master[census_master["codigo_comuna"].duplicated(keep=False)]
    .sort_values("codigo_comuna")
)

print("Duplicados actuales en EN0:")
display(
    eno_master[eno_master["codigo_comuna"].duplicated(keep=False)]
    .sort_values("codigo_comuna")
)

print("Duplicados actuales en GRD:")
display(
    grd_master[grd_master["codigo_comuna"].duplicated(keep=False)]
    .sort_values("codigo_comuna")
)

# También revisamos el número de filas y comunas únicas después de la corrección
print("Census - filas:", len(census_master), "| comunas únicas:", census_master["codigo_comuna"].nunique())
print("EN0 - filas:", len(eno_master), "| comunas únicas:", eno_master["codigo_comuna"].nunique())
print("GRD - filas:", len(grd_master), "| comunas únicas:", grd_master["codigo_comuna"].nunique())
```

Duplicados actuales en Census:

	codigo_comuna	nombre_comuna	pop_total	pop_chilean	pop_foreign	pct_foreign
5	13125	Quilicura	236478	214770	20141	8.57
6	13125	Tiltil	205624	177098	27310	13.36

Duplicados actuales en EN0:

	codigo_comuna	nombre_comuna	eno_total	eno_chilean	eno_foreign	eno_desconc
16	13125	Quilicura	1487	701	124	
17	13125	Tiltil	1592	769	270	

Duplicados actuales en GRD:

	codigo_comuna	nombre_comuna	grd_total	grd_chilean	grd_foreign	grd_pct_fore
25	13125	Quilicura	13062	11120	1942	14
26	13125	Tiltil	1435	1360	75	5

Census – filas: 37 | comunas únicas: 36

ENO – filas: 37 | comunas únicas: 36

GRD – filas: 37 | comunas únicas: 36

```
In [60]: # Corregimos manualmente el código de comuna de Tiltil

census_master.loc[
    census_master["nombre_comuna"].str.upper().str.contains("TILTIL", na=False)
    & census_master["codigo_comuna"] == 13303
] = 13303

eno_master.loc[
    eno_master["nombre_comuna"].str.upper().str.contains("TILTIL", na=False)
    & eno_master["codigo_comuna"] == 13303
] = 13303

grd_master.loc[
    grd_master["nombre_comuna"].str.upper().str.contains("TILTIL", na=False)
    & grd_master["codigo_comuna"] == 13303
] = 13303

# Revisamos nuevamente si quedan duplicados en cada tabla
print("Duplicados Census:", census_master["codigo_comuna"].duplicated().sum())
print("Duplicados ENO:", eno_master["codigo_comuna"].duplicated().sum())
print("Duplicados GRD:", grd_master["codigo_comuna"].duplicated().sum())

# Revisamos que Quilicura y Tiltil ahora tengan códigos distintos
display(census_master[census_master["nombre_comuna"].str.upper().str.contains("QUI")])
display(eno_master[eno_master["nombre_comuna"].str.upper().str.contains("QUI")])
display(grd_master[grd_master["nombre_comuna"].str.upper().str.contains("QUI")])
```

Duplicados Census: 0

Duplicados ENO: 0

Duplicados GRD: 0

	codigo_comuna	nombre_comuna	pop_total	pop_chilean	pop_foreign	pct_foreign
5	13125	Quilicura	236478	214770	20141	8.57
6	13303	Tiltil	205624	177098	27310	13.36

	codigo_comuna	nombre_comuna	eno_total	eno_chilean	eno_foreign	eno_desconoc
16	13125	Quilicura	1487	701	124	
17	13303	Tiltil	1592	769	270	

	codigo_comuna	nombre_comuna	grd_total	grd_chilean	grd_foreign	grd_pct_fore
25	13125	Quilicura	13062	11120	1942	14
26	13303	Tiltil	1435	1360	75	5

In [61]: `# PARTE 0.2: UNIÓN DE LAS BASES POR CODIGO_COMUNA`

```
# Eliminamos nombre_comuna de ENO y GRD para evitar columnas repetidas
# Mantendremos el nombre de comuna que viene desde Census
eno_clean = eno_master.drop(columns=["nombre_comuna"], errors="ignore")
grd_clean = grd_master.drop(columns=["nombre_comuna"], errors="ignore")

# Unimos Census con ENO usando codigo_comuna como llave
# how="inner" conserva solo las comunas que están en ambas bases
census_eno = census_master.merge(
    eno_clean,
    on="codigo_comuna",
    how="inner"
)

# Unimos la base anterior con GRD usando nuevamente codigo_comuna
analytical = census_eno.merge(
    grd_clean,
    on="codigo_comuna",
    how="inner"
```

```

)

# Mostramos el tamaño de las bases después de cada unión
print("Tamaño base Census + EN0:", census_eno.shape)
print("Tamaño base final Census + EN0 + GRD:", analytical.shape)

# Revisamos cuántas comunas únicas quedaron en la base final
print("Comunas únicas en base final:", analytical["codigo_comuna"].nunique())

# Mostramos una vista resumida para confirmar que quedaron columnas de las t
display(analytical[[
    "codigo_comuna",
    "nombre_comuna",
    "pop_total",
    "pct_foreign",
    "eno_total",
    "eno_rate_per_10k",
    "grd_total",
    "grd_rate_per_10k"
]].head())

```

Tamaño base Census + EN0: (37, 19)

Tamaño base final Census + EN0 + GRD: (37, 30)

Comunas únicas en base final: 37

	codigo_comuna	nombre_comuna	pop_total	pct_foreign	eno_total	eno_rate_per_10
0	13122	Peñalolén	236478	7.740255	1487	62.880
1	13605	Peñaflor	94402	3.426834	441	46.720
2	13114	LAS CONDES	292739	10.613208	2123	72.521
3	13503	CURACAVÍ	34977	4.174172	165	47.173
4	13109	La Reina	103157	19.380000	714	69.210

In [62]: # PARTE 0.3: CREACIÓN DE VARIABLES DERIVADAS

```

# Creamos una copia de la base integrada para trabajar sin modificar la orig
analytical = analytical.copy()

# Creamos el logaritmo natural de la población total
# Esta variable se usará después como offset en los modelos de conteo
analytical["log_pop_total"] = np.log(analytical["pop_total"])

# Calculamos una tasa de empleo combinada para la comuna
# Se pondera la tasa de empleo chilena y extranjera según el tamaño de cada
analytical["combined_emp_rate"] = (
    analytical["emp_rate_chilean"] * analytical["pop_chilean"] +
    analytical["emp_rate_foreign"] * analytical["pop_foreign"]
) / analytical["pop_total"]

# Calculamos el porcentaje de desempleo como complemento de la tasa de emple
# Como las tasas vienen en porcentaje, usamos 100 - tasa de empleo

```

```

analytical["pct_unemployed"] = 100 - analytical["combined_emp_rate"]

# Calculamos la brecha de escolaridad
# Es la diferencia entre escolaridad promedio chilena y extranjera
analytical["schooling_gap"] = (
    analytical["mean_schooling_chilean"] -
    analytical["mean_schooling_foreign"]
)

# Usamos dependency_ratio como variable adicional
# Esta variable ya venía en Census y representa la razón de dependencia demográfica
variable_adicional = "dependency_ratio"

# Mostramos las variables creadas para revisar que quedaron bien
display(analytical[[
    "codigo_comuna",
    "nombre_comuna",
    "pop_total",
    "pct_foreign",
    "log_pop_total",
    "combined_emp_rate",
    "pct_unemployed",
    "mean_schooling_chilean",
    "mean_schooling_foreign",
    "schooling_gap",
    variable_adicional
]].head())

```

	codigo_comuna	nombre_comuna	pop_total	pct_foreign	log_pop_total	combined_e
0	13122	Peñalolén	236478	7.740255	12.373610	64
1	13605	Peñaflor	94402	3.426834	11.455318	59
2	13114	LAS CONDES	292739	10.613208	12.587037	70
3	13503	CURACAVÍ	34977	4.174172	10.462446	6
4	13109	La Reina	103157	19.380000	11.544007	6

In [63]: # Guardamos la base analítica final

```

os.makedirs("output", exist_ok=True)

analytical.to_csv(
    "output/tarea3_analytical_table.csv",
    index=False
)

print("Archivo guardado correctamente en: output/tarea3_analytical_table.csv")
print("Tamaño final de la base:", analytical.shape)
print("Comunas únicas:", analytical["codigo_comuna"].nunique())

```

Archivo guardado correctamente en: output/tarea3_analytical_table.csv

Tamaño final de la base: (37, 34)

Comunas únicas: 37

In [72]: *# PARTE 1.1: MATRIZ DE CORRELACIÓN*

```
# Seleccionamos las variables que usaremos en la matriz de correlación
# Incluimos variables censales y los dos outcomes principales de salud
corr_vars = [
    "pct_foreign",
    "log_pop_total",
    "pct_unemployed",
    "schooling_gap",
    "mean_schooling_chilean",
    "mean_schooling_foreign",
    "dependency_ratio",
    "eno_rate_per_10k",
    "grd_rate_per_10k"
]

# Creamos una tabla solo con esas variables
# dropna elimina filas con datos faltantes para que no falle el cálculo
corr_df = analytical[corr_vars].dropna()

# Calculamos la matriz de correlación de Pearson
corr_matrix = corr_df.corr(method="pearson")

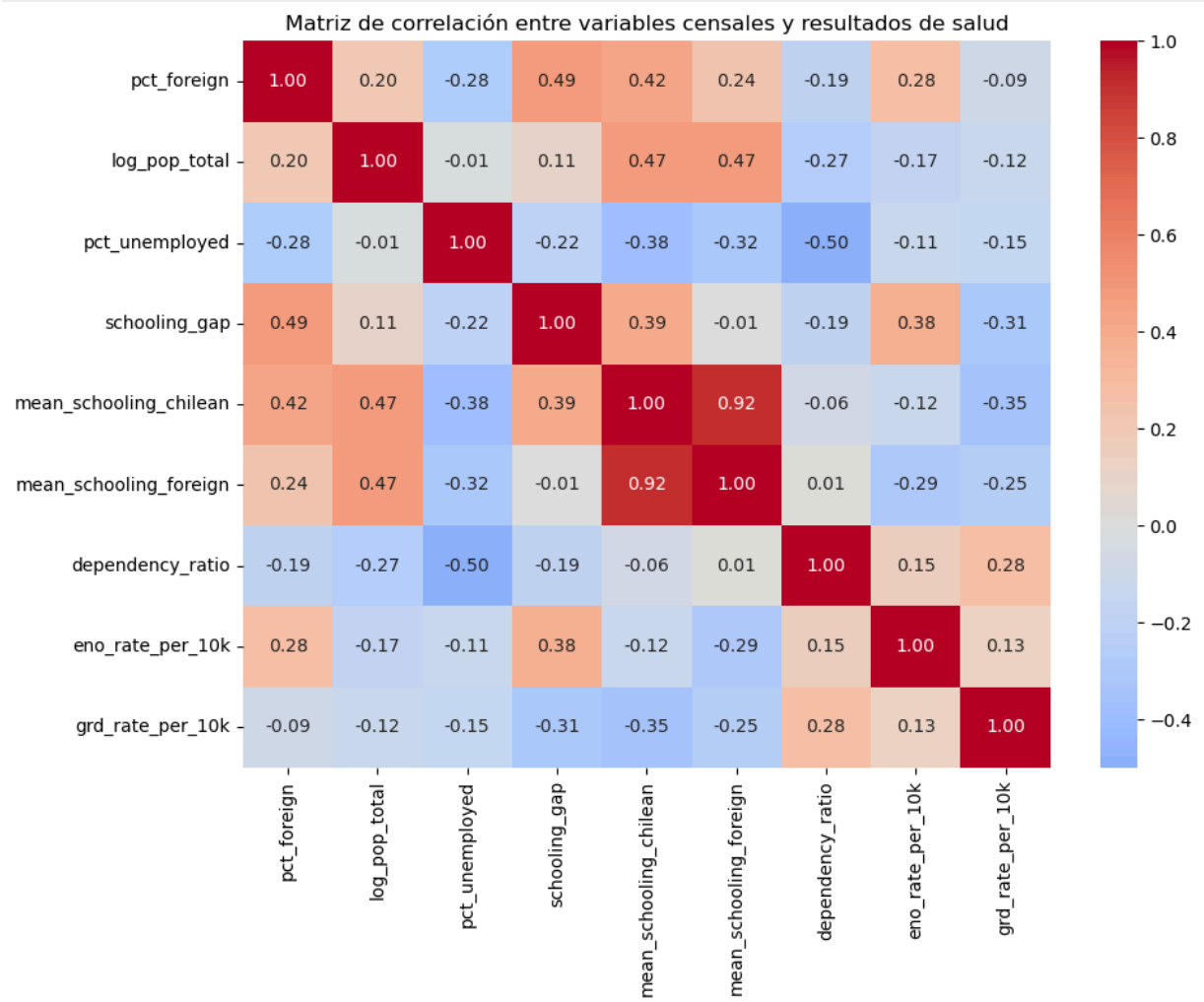
# Mostramos la matriz como tabla
display(corr_matrix)

# Graficamos la matriz como heatmap
plt.figure(figsize=(10, 8))

sns.heatmap(
    corr_matrix,
    annot=True,           # muestra los valores dentro de cada celda
    cmap="coolwarm",      # colores para correlaciones negativas y positivas
    center=0,             # centra la escala de color en cero
    fmt=".2f"              # muestra los valores con dos decimales
)

# Agregamos título y ajustamos el espacio del gráfico
plt.title("Matriz de correlación entre variables censales y resultados de sa
plt.tight_layout()
plt.show()
```

	pct_foreign	log_pop_total	pct_unemployed	schooling_gap	n
pct_foreign	1.000000	0.198800	-0.277123	0.488670	
log_pop_total	0.198800	1.000000	-0.011945	0.112629	
pct_unemployed	-0.277123	-0.011945	1.000000	-0.217436	
schooling_gap	0.488670	0.112629	-0.217436	1.000000	
mean_schooling_chilean	0.415558	0.474725	-0.379516	0.392823	
mean_schooling_foreign	0.239691	0.467305	-0.318269	-0.007050	
dependency_ratio	-0.186718	-0.267412	-0.499870	-0.190642	
eno_rate_per_10k	0.282446	-0.171869	-0.113211	0.379409	
grd_rate_per_10k	-0.094616	-0.117781	-0.150350	-0.310439	



In []: La matriz de correlación muestra asociaciones comunales entre variables demo. Se observa una correlación positiva muy alta entre `pct\_foreign`, `eno\_rate\_per\_10k`, sugiere que las comunas con mayor proporción de población extranjera tienden a tener una mayor tasa de envejecimiento. También aparece una correlación negativa fuerte entre `pct\_unemployed` y las `mean\_schooling\_foreign` están altamente correlacionadas, lo que indica que

chilena también tienden a presentar mayor escolaridad promedio en población asociaciones ecológicas a nivel comunal, no como efectos individuales.

```
In [67]: import statsmodels.api as sm
import statsmodels.formula.api as smf
```

```
In [74]: # PARTE 1.2: GRÁFICOS DE DISPERSIÓN BIVARIADOS

# Definimos las variables censales que irán en el eje X
# Estas variables se compararán con los outcomes de salud
x_vars = [
    "pct_foreign",
    "pct_unemployed",
    "schooling_gap",
    "mean_schooling_chilean",
    "dependency_ratio"
]

# Definimos los outcomes de salud que irán en el eje Y
y_vars = [
    "eno_rate_per_10k",
    "grd_rate_per_10k"
]

# Creamos gráficos de dispersión para cada outcome de salud
for y in y_vars:

    # Creamos una fila de gráficos, uno por cada variable censal
    fig, axes = plt.subplots(
        1,
        len(x_vars),
        figsize=(5 * len(x_vars), 4)
    )

    # Recorremos cada variable censal y su respectivo eje
    for ax, x in zip(axes, x_vars):

        # Dibujamos el scatter plot con una línea de regresión simple
        sns.regplot(
            data=analytical,
            x=x,
            y=y,
            ax=ax,
            scatter_kws={"s": 45}
        )

        # Creamos una tabla temporal sin valores faltantes
        temp = analytical[[x, y, "nombre_comuna"]].dropna().copy()

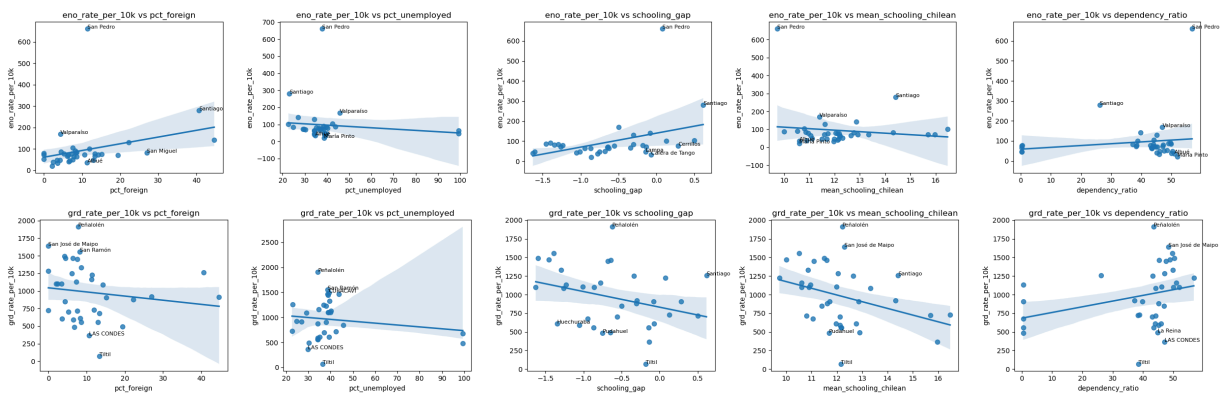
        # Ajustamos una regresión simple para calcular residuos
        # Esto permite etiquetar las comunas más alejadas de la línea
        X = sm.add_constant(temp[x])
        model = sm.OLS(temp[y], X).fit()
        temp["abs_resid"] = abs(model.resid)
```

```
# Seleccionamos las 5 comunas con residuos más extremos
extremos = temp.nlargest(5, "abs_resid")

# Agregamos etiquetas solo a esas comunas extremas
for _, row in extremos.iterrows():
    ax.text(
        row[x],
        row[y],
        row["nombre_comuna"],
        fontsize=8
    )

# Título de cada gráfico
ax.set_title(f"{y} vs {x}")

# Ajustamos espacios entre gráficos
plt.tight_layout()
plt.show()
```



In [69]: *# Revisamos valores extremos en variables clave*
Esto sirve para detectar posibles errores de escala o valores anómalos

```
variables_revision = [
    "nombre_comuna",
    "codigo_comuna",
    "pop_total",
    "pct_foreign",
    "combined_emp_rate",
    "pct_unemployed",
    "eno_rate_per_10k",
    "grd_rate_per_10k"
]

# Ordenamos por pct_foreign para ver comunas con valores demasiado altos
print("Comunas con mayor pct_foreign:")
display(
    analytical[variables_revision]
    .sort_values("pct_foreign", ascending=False)
    .head(10)
)

# Ordenamos por pct_unemployed para ver valores muy altos o negativos
print("Comunas con menor pct_unemployed:")
```



```

display(
  analytical[variables_revision]
  .sort_values("pct_unemployed", ascending=True)
  .head(10)
)

# Ordenamos por tasas de salud para ver outliers
print("Comunas con mayor eno_rate_per_10k:")
display(
  analytical[variables_revision]
  .sort_values("eno_rate_per_10k", ascending=False)
  .head(10)
)

print("Comunas con mayor grd_rate_per_10k:")
display(
  analytical[variables_revision]
  .sort_values("grd_rate_per_10k", ascending=False)
  .head(10)
)

```

Comunas con mayor pct_foreign:

	nombre_comuna	codigo_comuna	pop_total	pct_foreign	combined_emp_rate	pct
28	Santiago	13101	5806	3047.812608	5774.146634	
32	Independencia	13108	116943	44.387437	72.589764	
8	San Miguel	13130	150829	26.962985	75.221826	
33	Recoleta	13127	154615	21.995925	65.088421	
4	La Reina	13109	103157	19.380000	68.907871	
18	Cerrillos	13102	85041	15.000000	61.499827	
11	Macul	13118	123800	14.133279	70.303183	
6	Tiltil	13303	205624	13.360000	63.313260	
21	Lo Prado	13117	91290	13.265418	0.639479	
20	Lampa	13302	145160	12.800000	65.250384	

Comunas con menor pct_unemployed:

	nombre_comuna	codigo_comuna	pop_total	pct_foreign	combined_emp_rate	pct
28	Santiago	13101	5806	3047.812608	5774.146634	
7	Providencia	13123	143974	11.900065	77.356569	
8	San Miguel	13130	150829	26.962985	75.221826	
32	Independencia	13108	116943	44.387437	72.589764	
11	Macul	13118	123800	14.133279	70.303183	
2	LAS CONDES	13114	292739	10.613208	70.289464	
4	La Reina	13109	103157	19.380000	68.907871	
35	Padre Hurtado	13604	81243	6.518715	65.380642	
0	Peñalolén	13122	236478	7.740255	65.300789	
5	Quilicura	13125	236478	8.570000	65.296187	

Comunas con mayor eno_rate_per_10k:

	nombre_comuna	codigo_comuna	pop_total	pct_foreign	combined_emp_rate	pct
28	Santiago	13101	5806	3047.812608	5774.146634	
34	San Pedro	13505	11108	11.343176	61.934657	
13	Valparaíso	5101	284938	4.320000	54.302109	
32	Independencia	13108	116943	44.387437	72.589764	
8	San Miguel	13130	150829	26.962985	75.221826	
14	Casablanca	5102	29876	7.590000	57.569588	
33	Recoleta	13127	154615	21.995925	65.088421	
7	Providencia	13123	143974	11.900065	77.356569	
12	San Ramón	13131	76002	8.176101	61.001201	
24	La Pintana	13112	175421	4.440000	55.854599	

Comunas con mayor grd_rate_per_10k:

	nombre_comuna	codigo_comuna	pop_total	pct_foreign	combined_emp_rate	pct
28	Santiago	13101	5806	3047.812608	5774.146634	
0	Peñalolén	13122	236478	7.740255	65.300789	
17	San José de Maipo	13203	17441	0.000000	NaN	
9	Paine	13404	78828	7.564571	61.251640	
12	San Ramón	13131	76002	8.176101	61.001201	
3	CURACAVÍ	13503	34977	4.174172	60.791202	
24	La Pintana	13112	175421	4.440000	55.854599	
29	San Bernardo	13401	306371	6.380000	61.040725	
10	La Granja	13111	112022	8.525111	60.040474	
8	San Miguel	13130	150829	26.962985	75.221826	

In []:

```

In [70]: # LIMPIEZA DE DATOS: RECALCULAR POBLACIÓN, PORCENTAJES Y TASAS

# Recalculamos la población total desde población chilena + extranjera
# Esto corrige casos donde pop_total viene mal escalado, como Santiago
analytical["pop_total"] = analytical["pop_chilean"] + analytical["pop_foreign"]

# Recalculamos el porcentaje de población extranjera
# Así evitamos valores imposibles como porcentajes mayores a 100
analytical["pct_foreign"] = (
    analytical["pop_foreign"] / analytical["pop_total"]
) * 100

# Recalculamos el logaritmo de la población total corregida
analytical["log_pop_total"] = np.log(analytical["pop_total"])

# Recalculamos la tasa ENO por cada 10.000 habitantes
analytical["eno_rate_per_10k"] = (
    analytical["eno_total"] / analytical["pop_total"]
) * 10000

# Recalculamos la tasa GRD por cada 10.000 habitantes
analytical["grd_rate_per_10k"] = (
    analytical["grd_total"] / analytical["pop_total"]
) * 10000

# Recalculamos la tasa de empleo combinada
analytical["combined_emp_rate"] = (
    analytical["emp_rate_chilean"] * analytical["pop_chilean"] +
    analytical["emp_rate_foreign"] * analytical["pop_foreign"]
)

```

```

) / analytical["pop_total"]

# Recalculamos el porcentaje de desempleo
analytical["pct_unemployed"] = 100 - analytical["combined_emp_rate"]

# Recalculamos brecha de escolaridad
analytical["schooling_gap"] = (
    analytical["mean_schooling_chilean"] -
    analytical["mean_schooling_foreign"]
)

# Revisamos nuevamente valores extremos
variables_revision = [
    "nombre_comuna",
    "codigo_comuna",
    "pop_total",
    "pct_foreign",
    "combined_emp_rate",
    "pct_unemployed",
    "eno_rate_per_10k",
    "grd_rate_per_10k"
]

display(
    analytical[variables_revision]
    .sort_values("pct_foreign", ascending=False)
    .head(10)
)

```

	nombre_comuna	codigo_comuna	pop_total	pct_foreign	combined_emp_rate	pct_
32	Independencia	13108	116180	44.678946	73.066489	
28	Santiago	13101	435385	40.643568	77.000116	
8	San Miguel	13130	150829	26.962985	75.221826	
33	Recoleta	13127	152977	22.231447	65.785355	
4	La Reina	13109	101971	19.380020	69.709322	
18	Cerrillos	13102	84198	15.121499	62.115570	
11	Macul	13118	122833	14.244543	70.856644	
6	Tiltil	13303	204408	13.360534	63.689904	
21	Lo Prado	13117	90788	13.338767	0.643015	
20	Lampa	13302	144094	12.913792	65.733103	

In [71]: # Revisamos nuevamente valores extremos después de corregir población y tasa

```

print("Comunas con mayor pct_foreign:")
display(
    analytical[variables_revision]

```

```

        .sort_values("pct_foreign", ascending=False)
        .head(10)
    )

    print("Comunas con menor pct_unemployed:")
    display(
        analytical[variables_revision]
        .sort_values("pct_unemployed", ascending=True)
        .head(10)
    )

    print("Comunas con mayor eno_rate_per_10k:")
    display(
        analytical[variables_revision]
        .sort_values("eno_rate_per_10k", ascending=False)
        .head(10)
    )

    print("Comunas con mayor grd_rate_per_10k:")
    display(
        analytical[variables_revision]
        .sort_values("grd_rate_per_10k", ascending=False)
        .head(10)
    )

```

Comunas con mayor pct_foreign:

	nombre_comuna	codigo_comuna	pop_total	pct_foreign	combined_emp_rate	pct_
32	Independencia	13108	116180	44.678946	73.066489	
28	Santiago	13101	435385	40.643568	77.000116	
8	San Miguel	13130	150829	26.962985	75.221826	
33	Recoleta	13127	152977	22.231447	65.785355	
4	La Reina	13109	101971	19.380020	69.709322	
18	Cerrillos	13102	84198	15.121499	62.115570	
11	Macul	13118	122833	14.244543	70.856644	
6	Tiltil	13303	204408	13.360534	63.689904	
21	Lo Prado	13117	90788	13.338767	0.643015	
20	Lampa	13302	144094	12.913792	65.733103	

Comunas con menor pct_unemployed:

	nombre_comuna	codigo_comuna	pop_total	pct_foreign	combined_emp_rate	pct_
7	Providencia	13123	143974	11.900065	77.356569	
28	Santiago	13101	435385	40.643568	77.000116	
8	San Miguel	13130	150829	26.962985	75.221826	
32	Independencia	13108	116180	44.678946	73.066489	
11	Macul	13118	122833	14.244543	70.856644	
2	LAS CONDES	13114	292739	10.613208	70.289464	
4	La Reina	13109	101971	19.380020	69.709322	
33	Recoleta	13127	152977	22.231447	65.785355	
35	Padre Hurtado	13604	80793	6.555023	65.744798	
20	Lampa	13302	144094	12.913792	65.733103	

Comunas con mayor eno_rate_per_10k:

	nombre_comuna	codigo_comuna	pop_total	pct_foreign	combined_emp_rate	pct_
34	San Pedro	13505	11068	11.384171	62.158490	
28	Santiago	13101	435385	40.643568	77.000116	
13	Valparaíso	5101	284938	4.323748	54.302109	
32	Independencia	13108	116180	44.678946	73.066489	
33	Recoleta	13127	152977	22.231447	65.785355	
14	Casablanca	5102	29876	7.594725	57.569588	
7	Providencia	13123	143974	11.900065	77.356569	
12	San Ramón	13131	75462	8.234608	61.437720	
24	La Pintana	13112	174215	4.438194	56.241252	
10	La Granja	13111	111255	8.583884	60.454397	

Comunas con mayor grd_rate_per_10k:

	nombre_comuna	codigo_comuna	pop_total	pct_foreign	combined_emp_rate	pct_
0	Peñalolén	13122	234968	7.789997	65.720439	
17	San José de Maipo	13203	17303	0.000000	NaN	
12	San Ramón	13131	75462	8.234608	61.437720	
3	CURACAVÍ	13503	34977	4.174172	60.791202	
24	La Pintana	13112	174215	4.438194	56.241252	
29	San Bernardo	13401	304207	6.424573	61.474943	
9	Paine	13404	78828	7.564571	61.251640	
10	La Granja	13111	111255	8.583884	60.454397	
16	San Joaquín	13129	95039	0.000000	NaN	
28	Santiago	13101	435385	40.643568	77.000116	

```
In [73]: # Recalculamos la tabla de correlación usando los datos ya corregidos
corr_df = analytical[corr_vars].dropna()

# Calculamos la matriz de correlación de Pearson
corr_matrix = corr_df.corr(method="pearson")

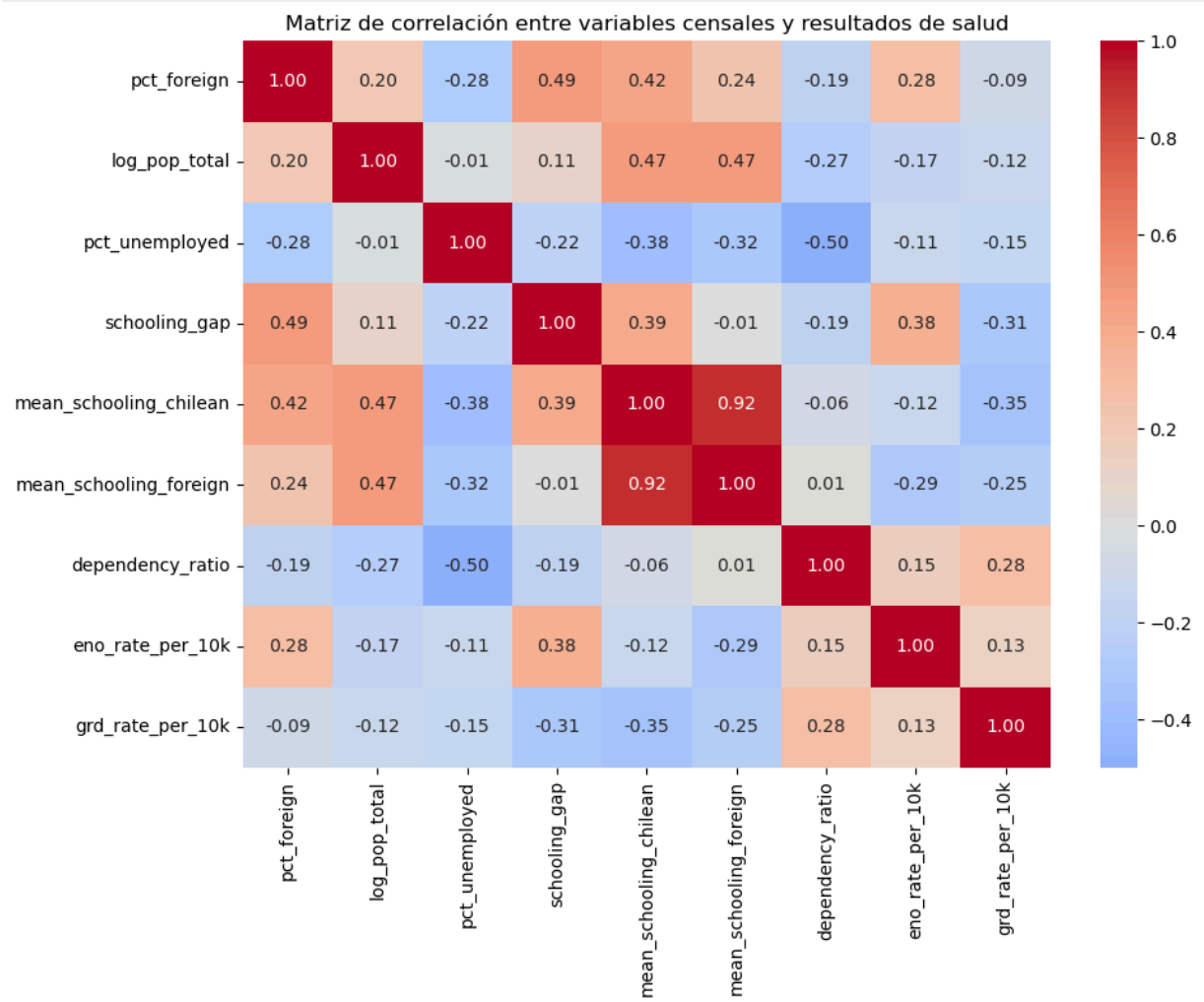
# Mostramos la matriz como tabla
display(corr_matrix)

# Graficamos la matriz como heatmap
plt.figure(figsize=(10, 8))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    center=0,
    fmt=".2f"
)

plt.title("Matriz de correlación entre variables censales y resultados de sa
plt.tight_layout()
plt.show()
```

	pct_foreign	log_pop_total	pct_unemployed	schooling_gap	n
pct_foreign	1.000000	0.198800	-0.277123	0.488670	
log_pop_total	0.198800	1.000000	-0.011945	0.112629	
pct_unemployed	-0.277123	-0.011945	1.000000	-0.217436	
schooling_gap	0.488670	0.112629	-0.217436	1.000000	
mean_schooling_chilean	0.415558	0.474725	-0.379516	0.392823	
mean_schooling_foreign	0.239691	0.467305	-0.318269	-0.007050	
dependency_ratio	-0.186718	-0.267412	-0.499870	-0.190642	
eno_rate_per_10k	0.282446	-0.171869	-0.113211	0.379409	
grd_rate_per_10k	-0.094616	-0.117781	-0.150350	-0.310439	



In []: Se detectó un problema de escala en la comuna de Santiago, donde `pop\_total` generaba porcentajes y tasas artificialmente extremos. Para corregirlo, se re suma de `pop\_chilean` y `pop\_foreign`, y luego se recalcularon `pct\_foreign`, `log\_pop\_total`, `combined\_emp\_rate`, `pct\_unemployed` y `schooling\_gap`. Es distorsione la matriz de correlación, los gráficos y los modelos posteriores

```
In [75]: # PARTE 1.3: IDENTIFICACIÓN DE OUTLIERS BIVARIADOS

# Esta función calcula los residuos de una regresión lineal simple
# Sirve para encontrar las comunas que más se alejan de la línea de tendencia
def detectar_outliers_bivariados(df, x, y, n=5):

    # Nos quedamos solo con las columnas necesarias y eliminamos valores faltantes
    temp = df[[x, y, "nombre_comuna"]].dropna().copy()

    # Agregamos una constante para ajustar el modelo OLS
    X = sm.add_constant(temp[x])

    # Ajustamos una regresión lineal simple entre x e y
    model = sm.OLS(temp[y], X).fit()

    # Calculamos los residuos del modelo
    temp["residuo"] = model.resid

    # Calculamos el valor absoluto del residuo para ordenar por distancia a la línea
    temp["abs_residuo"] = temp["residuo"].abs()

    # Retornamos las comunas con residuos más grandes
    return temp.sort_values("abs_residuo", ascending=False).head(n)

# Detectamos outliers para ENO usando pct_foreign como variable explicativa
outliers_eno = detectar_outliers_bivariados(
    analytical,
    x="pct_foreign",
    y="eno_rate_per_10k",
    n=5
)

# Detectamos outliers para GRD usando pct_foreign como variable explicativa
outliers_grd = detectar_outliers_bivariados(
    analytical,
    x="pct_foreign",
    y="grd_rate_per_10k",
    n=5
)

print("Outliers ENO vs pct_foreign:")
display(outliers_eno)

print("Outliers GRD vs pct_foreign:")
display(outliers_grd)
```

Outliers ENO vs pct_foreign:

	pct_foreign	eno_rate_per_10k	nombre_comuna	residuo	abs_residuo
34	11.384171	660.462595	San Pedro	563.552802	563.552802
13	4.323748	169.194702	Valparaíso	94.402711	94.402711
28	40.643568	279.890212	Santiago	91.321098	91.321098
8	26.962985	82.809009	San Miguel	-62.903686	62.903686
25	11.341935	36.129032	Alhué	-60.648453	60.648453

Outliers GRD vs pct_foreign:

	pct_foreign	grd_rate_per_10k	nombre_comuna	residuo	abs_residuo
0	7.789997	1911.622008	Peñalolén	913.314374	913.314374
6	13.360534	70.202732	Tiltil	-895.391084	895.391084
2	10.613208	364.010262	LAS CONDES	-617.717637	617.717637
17	0.000000	1643.645611	San José de Maipo	599.590044	599.590044
12	8.234608	1556.147465	San Ramón	560.450877	560.450877

In []: En los gráficos bivariados se observan algunas comunas que se alejan visualmente de la tendencia general. En el caso de ENO, San Pedro aparece como una comuna con una tasa considerablemente alta. Esto podría deberse a un tamaño poblacional menor combinado con un número relevante de egresos. También podría reflejar diferencias territoriales en la exposición o en la frecuencia de enfermedades notificables.

En el caso de GRD, Peñalolén destaca por una tasa alta en comparación con otras comunas. Una posible explicación es que la comuna concentre una demanda hospitalaria alta asociada a una población que utiliza servicios de salud de manera distinta a otras comunas. Tiltil aparece con una tasa GRD muy baja, lo que podría estar relacionado con una menor demanda de egresos de la comuna o menor registro de egresos asociados al territorio. Estas hipótesis requieren ser validadas.

In []: En el análisis bivariado entre `pct\_foreign` y `eno\_rate\_per\_10k`, San Pedro aparece como el principal outlier, con un residuo positivo muy alto. Esto significa que su tasa de egresos es mayor a la que sugeriría la tendencia general con respecto al porcentaje de extranjeros. Una posible explicación es que, al tratarse de una comuna con menor población, un número relativamente pequeño de egresos puede generar una tasa por 10.000 habitantes muy elevada. También podría existir una mayor incidencia de enfermedades notificables.

En el caso de `grd\_rate\_per\_10k`, Peñalolén aparece como un outlier positivo muy relevante. Peñalolén presenta una tasa GRD mayor a la que podría estar asociado a una mayor concentración de egresos hospitalarios. Esto podría deberse a una mayor demanda de servicios de salud o a una mayor proporción de egresos de alta complejidad.

En cambio, Tiltil muestra una tasa GRD mucho menor a la esperada, lo que podría deberse a una menor demanda de egresos fuera de la comuna o diferencias en el registro de egresos. Estas hipótesis requieren ser validadas para afirmar causalidad.

```
In [76]: # PARTE 2.1: MODELO POISSON PARA ENO

# Importamos statsmodels para modelos estadísticos
import statsmodels.api as sm
import statsmodels.formula.api as smf
```

```

# Definimos las covariables que usaremos en el modelo
# Incluimos población extranjera, educación, empleo y dependencia demográfica
covariates = [
    "pct_foreign",
    "mean_schooling_chilean",
    "pct_unemployed",
    "dependency_ratio"
]

# Creamos una base solo con las columnas necesarias para el modelo EN0
# eno_total será la variable dependiente
# pop_total se usará como offset para modelar tasas y no solo conteos brutos
model_eno_df = analytical[
    ["nombre_comuna", "eno_total", "pop_total"] + covariates
].dropna().copy()

# Revisamos cuántas observaciones quedan después de eliminar valores faltantes
print("Filas disponibles para modelo EN0:", len(model_eno_df))
print("Covariables usadas:", covariates)

# Mostramos las primeras filas de la base del modelo
display(model_eno_df.head())

```

Filas disponibles para modelo EN0: 34

Covariables usadas: ['pct_foreign', 'mean_schooling_chilean', 'pct_unemployed', 'dependency_ratio']

	nombre_comuna	eno_total	pop_total	pct_foreign	mean_schooling_chilean	pct_unemployed
0	Peñalolén	1487	234968	7.789997	12.222864	3.0
1	Peñaflor	441	93851	3.446953	11.981959	3.0
2	LAS CONDES	2123	292739	10.613208	15.967253	2.0
3	CURACAVÍ	165	34977	4.174172	11.494170	3.0
4	La Reina	714	101971	19.380020	12.910000	3.0

```

In [77]: # Ajustamos el modelo Poisson para EN0
# La variable dependiente es eno_total, es decir, el conteo de notificaciones

formula_eno = "eno_total ~ " + " + ".join(covariates)

# Ajustamos un modelo GLM Poisson
# El offset usa log(pop_total), para modelar tasas ajustadas por población
poisson_eno = smf.glm(
    formula=formula_eno,
    data=model_eno_df,
    family=sm.families.Poisson(),
    offset=np.log(model_eno_df["pop_total"])
).fit()

# Mostramos el resumen completo del modelo
print(poisson_eno.summary())

```

Generalized Linear Model Regression Results

=====					
==					
Dep. Variable:		eno_total	No. Observations:		
34					
Model:		GLM	Df Residuals:		
29					
Model Family:		Poisson	Df Model:		
4					
Link Function:		Log	Scale:		1.00
00					
Method:		IRLS	Log-Likelihood:		-414
2.1					
Date:		Sun, 03 May 2026	Deviance:		799
4.8					
Time:		22:00:45	Pearson chi2:		1.05e+
04					
No. Iterations:		5	Pseudo R-squ. (CS):		1.0
00					
Covariance Type:		nonrobust			
=====					
=====					
		coef	std err	z	P> z
25					[0.0
0.975]					

Intercept		-4.8378	0.065	-74.874	0.000
64					-4.9
-4.711					
pct_foreign		0.0346	0.000	81.249	0.000
34					0.0
0.035					
mean_schooling_chilean		-0.0348	0.004	-9.133	0.000
42					-0.0
-0.027					
pct_unemployed		-0.0017	0.000	-3.601	0.000
03					-0.0
-0.001					
dependency_ratio		0.0050	0.000	12.049	0.000
04					0.0
0.006					
=====					
=====					

```
In [78]: # Creamos una función para transformar los coeficientes del modelo en IRR
# IRR = exp(coeficiente), y sirve para interpretar el efecto sobre la tasa e

def crear_tabla_irr(modelo):

    # Extraemos los coeficientes del modelo
    coef = modelo.params

    # Extraemos los intervalos de confianza de los coeficientes
    conf = modelo.conf_int()

    # Creamos una tabla resumen
    tabla = pd.DataFrame({
        "coef": coef,
        "std_error": modelo.bse,
        "p_value": modelo.pvalues,
        "IRR": np.exp(coef),
```

```

        "IRR_2.5%": np.exp(conf[0]),
        "IRR_97.5%": np.exp(conf[1])
    })

    return tabla

# Aplicamos la función al modelo Poisson de ENO
irr_eno_poisson = crear_tabla_irr(poisson_eno)

# Mostramos la tabla IRR
display(irr_eno_poisson)

```

	coef	std_error	p_value	IRR	IRR_2.5%	IRR_97.5%
Intercept	-4.837847	0.064614	0.000000e+00	0.007924	0.006982	0.010000
pct_foreign	0.034640	0.000426	0.000000e+00	1.035247	1.034382	1.036112
mean_schooling_chilean	-0.034836	0.003814	6.636090e-20	0.965764	0.958572	0.973000
pct_unemployed	-0.001670	0.000464	3.166075e-04	0.998332	0.997425	0.999240
dependency_ratio	0.004974	0.000413	1.970713e-33	1.004986	1.004174	1.005800

```

In [79]: # Reportamos medidas de ajuste del modelo Poisson
# Estas métricas sirven para evaluar el ajuste general del modelo

print("Deviance:", poisson_eno.deviance)
print("Pearson chi-square:", poisson_eno.pearson_chi2)
print("AIC:", poisson_eno.aic)
print("Grados de libertad residuales:", poisson_eno.df_resid)

```

Deviance: 7994.844693436862
 Pearson chi-square: 10475.742428246807
 AIC: 8294.185243876664
 Grados de libertad residuales: 29

```

In [80]: # PARTE 2.2: REVISIÓN DE SOBREDISPERSIÓN

# Calculamos la estadística de dispersión
# Si el valor está cerca de 1, Poisson puede ser razonable
# Si el valor es mucho mayor que 1, hay sobredispersión
dispersion_eno = poisson_eno.pearson_chi2 / poisson_eno.df_resid

print("Estadística de dispersión ENO:", dispersion_eno)

# Interpretamos automáticamente el resultado
if dispersion_eno > 2:
    print("Hay evidencia de sobredispersión. Conviene probar un modelo Negat
else:
    print("No hay evidencia fuerte de sobredispersión. Poisson podría ser su

```

Estadística de dispersión EN0: 361.23249752575197

Hay evidencia de sobredispersión. Conviene probar un modelo Negative Binomial.

In []: La estadística de dispersión **del** modelo Poisson para EN0 fue **361.23**, calcula los grados de libertad residuales. Este valor es muy superior a **1**, por lo que esto indica que la varianza observada en los conteos EN0 es mucho mayor que la media, lo que indica que los errores estándar **del** modelo Poisson probablemente están subestimados y el modelo principal.

```
In [83]: # =====
# PARTE 2.3: MODELO NEGATIVE BINOMIAL PARA EN0
# =====

# Ajustamos un modelo Negative Binomial usando las mismas covariables
# Este modelo es más flexible que Poisson cuando existe sobredispersión

nb_eno = smf.glm(
    formula=formula_eno,
    data=model_eno_df,
    family=sm.families.NegativeBinomial(),
    offset=np.log(model_eno_df["pop_total"])
).fit()

# Mostramos el resumen completo del modelo
print(nb_eno.summary())
```

Generalized Linear Model Regression Results

```

=====
==
Dep. Variable:          eno_total    No. Observations:
34
Model:                  GLM          Df Residuals:
29
Model Family:          NegativeBinomial    Df Model:
4
Link Function:          Log          Scale:                  1.00
00
Method:                  IRLS         Log-Likelihood:        -266.
48
Date:                   Sun, 03 May 2026    Deviance:              11.1
66
Time:                   22:04:50          Pearson chi2:           1
7.7
No. Iterations:         20              Pseudo R-squ. (CS):     0.16
87
Covariance Type:        nonrobust
=====
=====

```

	coef	std err	z	P> z	[0.0
25	0.975]				
Intercept	-4.0859	2.182	-1.873	0.061	-8.3
62	0.190				
pct_foreign	0.0434	0.021	2.058	0.040	0.0
02	0.085				
mean_schooling_chilean	-0.1349	0.137	-0.986	0.324	-0.4
03	0.133				
pct_unemployed	1.69e-05	0.015	0.001	0.999	-0.0
29	0.029				
dependency_ratio	0.0124	0.013	0.939	0.348	-0.0
14	0.038				

```

/opt/anaconda3/lib/python3.13/site-packages/statsmodels/genmod/families/fami
ly.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not s
et. Using default value alpha=1.0.
warnings.warn("Negative binomial dispersion parameter alpha not "

```

```

In [84]: # Creamos la tabla IRR para el modelo Negative Binomial
# Esto permite interpretar los coeficientes como cambios relativos en la tas

irr_eno_nb = crear_tabla_irr(nb_eno)

# Mostramos la tabla IRR del modelo Negative Binomial
display(irr_eno_nb)

```

	coef	std_error	p_value	IRR	IRR_2.5%	IRR_97.5
Intercept	-4.085896	2.181536	0.061076	0.016808	0.000234	1.2090
pct_foreign	0.043359	0.021064	0.039551	1.044313	1.002076	1.0883
mean_schooling_chilean	-0.134918	0.136794	0.323991	0.873788	0.668293	1.1424
pct_unemployed	0.000017	0.014792	0.999089	1.000017	0.971441	1.0294
dependency_ratio	0.012427	0.013239	0.347909	1.012504	0.986570	1.0391

In [85]: *# Creamos una tabla comparativa entre Poisson y Negative Binomial*
Nos interesa comparar coeficientes y p-values entre ambos modelos

```
comparacion_poisson_nb = pd.DataFrame({
    "coef_poisson": poisson_eno.params,
    "pvalue_poisson": poisson_eno.pvalues,
    "IRR_poisson": np.exp(poisson_eno.params),
    "coef_NB": nb_eno.params,
    "pvalue_NB": nb_eno.pvalues,
    "IRR_NB": np.exp(nb_eno.params)
})
```

```
# Mostramos la comparación
display(comparacion_poisson_nb)
```

	coef_poisson	pvalue_poisson	IRR_poisson	coef_NB	pvalue_NB
Intercept	-4.837847	0.000000e+00	0.007924	-4.085896	0.061076
pct_foreign	0.034640	0.000000e+00	1.035247	0.043359	0.039551
mean_schooling_chilean	-0.034836	6.636090e-20	0.965764	-0.134918	0.323991
pct_unemployed	-0.001670	3.166075e-04	0.998332	0.000017	0.999089
dependency_ratio	0.004974	1.970713e-33	1.004986	0.012427	0.347909

In [86]: *# Reportamos métricas de ajuste del modelo Negative Binomial*
Estas permiten comparar el ajuste general con el modelo Poisson

```
print("Deviance NB:", nb_eno.deviance)
print("Pearson chi-square NB:", nb_eno.pearson_chi2)
print("AIC NB:", nb_eno.aic)
print("Grados de libertad residuales NB:", nb_eno.df_resid)
```

```
Deviance NB: 11.166442385712262
Pearson chi-square NB: 17.72602687833128
AIC NB: 542.9536727031308
Grados de libertad residuales NB: 29
```


In []: Al comparar el modelo Poisson con el modelo Negative Binomial, se observa que las covariables como estadísticamente significativas. Sin embargo, el diagnóstico valor de 361.23, muy superior a 1, por lo que el supuesto Poisson de igualdad de varianzas no se cumple. Al ajustar el modelo Negative Binomial, los errores estándar aumentan y varían. En esta especificación, `pct\_foreign` se mantiene como una variable relevante, pero `pct\_unemployed` y `dependency\_ratio` pierden significancia estadística. Por lo tanto, es más apropiado como resultado principal para ENO.

```
In [87]: # PARTE 2.4: MODELO DE CONTEO PARA GRD

# Creamos una base solo con las columnas necesarias para el modelo GRD
# grd_total será la variable dependiente
# pop_total se usará como offset para modelar tasas ajustadas por población
model_grd_df = analytical[
    ["nombre_comuna", "grd_total", "pop_total"] + covariates
].dropna().copy()

# Revisamos cuántas observaciones quedan después de eliminar valores faltantes
print("Filas disponibles para modelo GRD:", len(model_grd_df))
print("Covariables usadas:", covariates)

# Mostramos las primeras filas de la base del modelo
display(model_grd_df.head())
```

Filas disponibles para modelo GRD: 34

Covariables usadas: ['pct_foreign', 'mean_schooling_chilean', 'pct_unemployed', 'dependency_ratio']

	nombre_comuna	grd_total	pop_total	pct_foreign	mean_schooling_chilean	pct_unemployed
0	Peñalolén	44917	234968	7.789997	12.222864	3.446953
1	Peñaflor	10357	93851	3.446953	11.981959	3.446953
2	LAS CONDES	10656	292739	10.613208	15.967253	2.927390
3	CURACAVÍ	5216	34977	4.174172	11.494170	3.446953
4	La Reina	5004	101971	19.380020	12.910000	3.446953

```
In [88]: # Ajustamos el modelo Poisson para GRD
# La variable dependiente es grd_total, es decir, el conteo de egresos GRD

formula_grd = "grd_total ~ " + " + ".join(covariates)

# Ajustamos un modelo GLM Poisson
# Usamos offset=np.log(pop_total) para modelar tasas ajustadas por población
poisson_grd = smf.glm(
    formula=formula_grd,
    data=model_grd_df,
    family=sm.families.Poisson(),
    offset=np.log(model_grd_df["pop_total"])
).fit()
```

```
# Mostramos el resumen del modelo
print(poisson_grd.summary())
```

Generalized Linear Model Regression Results

```
=====
==
Dep. Variable:          grd_total    No. Observations:
34
Model:                  GLM          Df Residuals:
29
Model Family:           Poisson      Df Model:
4
Link Function:          Log          Scale:                1.00
00
Method:                 IRLS         Log-Likelihood:        -4572
2.
Date:                   Sun, 03 May 2026    Deviance:            9107
1.
Time:                   22:08:08           Pearson chi2:         8.09e+
04
No. Iterations:         5               Pseudo R-squ. (CS):   1.0
00
Covariance Type:        nonrobust
=====
=====
```

	coef	std err	z	P> z	[0.0
25	0.975]				

Intercept	-0.2181	0.020	-10.836	0.000	-0.2
58	-0.179				
pct_foreign	0.0076	0.000	47.304	0.000	0.0
07	0.008				
mean_schooling_chilean	-0.1581	0.001	-124.214	0.000	-0.1
61	-0.156				
pct_unemployed	-0.0102	0.000	-67.870	0.000	-0.0
11	-0.010				
dependency_ratio	0.0030	0.000	25.533	0.000	0.0
03	0.003				

```
=====
=====
```

```
In [89]: # Calculamos la estadística de dispersión del modelo Poisson para GRD
# Si este valor es mucho mayor que 1, conviene usar Negative Binomial

dispersion_grd = poisson_grd.pearson_chi2 / poisson_grd.df_resid

print("Estadística de dispersión GRD:", dispersion_grd)

if dispersion_grd > 2:
    print("Hay evidencia de sobredispersión. Conviene usar Negative Binomial")
else:
    print("No hay evidencia fuerte de sobredispersión. Poisson podría ser su
```

Estadística de dispersión GRD: 2789.3577626049596

Hay evidencia de sobredispersión. Conviene usar Negative Binomial para GRD.

```
In [90]: # Ajustamos el modelo Negative Binomial para GRD
# Usamos las mismas covariables y el mismo offset de población

nb_grd = smf.glm(
    formula=formula_grd,
    data=model_grd_df,
    family=sm.families.NegativeBinomial(),
    offset=np.log(model_grd_df["pop_total"])
).fit()

# Mostramos el resumen del modelo Negative Binomial para GRD
print(nb_grd.summary())
```

Generalized Linear Model Regression Results

```
=====
==
Dep. Variable:          grd_total    No. Observations:
34
Model:                  GLM          Df Residuals:
29
Model Family:          NegativeBinomial    Df Model:
4
Link Function:          Log          Scale:          1.00
00
Method:                IRLS          Log-Likelihood: -347.
07
Date:                  Sun, 03 May 2026    Deviance:          6.90
49
Time:                  22:08:54          Pearson chi2:          4.
59
No. Iterations:          6          Pseudo R-squ. (CS):          0.044
54
Covariance Type:          nonrobust
=====
=====
coef      std err          z      P>|z|      [0.0
25      0.975]
-----
Intercept          -0.4835      2.180      -0.222      0.824      -4.7
56      3.789
pct_foreign          0.0040      0.021      0.190      0.849      -0.0
37      0.045
mean_schooling_chilean -0.1436      0.137      -1.051      0.293      -0.4
11      0.124
pct_unemployed          -0.0081      0.015      -0.550      0.582      -0.0
37      0.021
dependency_ratio          0.0032      0.013      0.238      0.812      -0.0
23      0.029
=====
=====
```

```
/opt/anaconda3/lib/python3.13/site-packages/statsmodels/genmod/families/fami
ly.py:1367: ValueWarning: Negative binomial dispersion parameter alpha not s
et. Using default value alpha=1.0.
warnings.warn("Negative binomial dispersion parameter alpha not "
```

```
In [91]: # Creamos la tabla IRR del modelo Negative Binomial para GRD
# Esto permite interpretar los efectos como cambios relativos en la tasa esp

irr_grd_nb = crear_tabla_irr(nb_grd)

# Mostramos la tabla IRR
display(irr_grd_nb)
```

	coef	std_error	p_value	IRR	IRR_2.5%	IRR_97.5%
Intercept	-0.483508	2.179979	0.824474	0.616617	0.008598	44.2197
pct_foreign	0.003998	0.021056	0.849426	1.004006	0.963414	1.0463
mean_schooling_chilean	-0.143598	0.136680	0.293435	0.866236	0.662665	1.1323
pct_unemployed	-0.008129	0.014783	0.582398	0.991904	0.963576	1.0210
dependency_ratio	0.003155	0.013229	0.811527	1.003160	0.977483	1.0295

```
In [92]: # Reportamos métricas de ajuste del modelo Negative Binomial para GRD

print("Deviance NB GRD:", nb_grd.deviance)
print("Pearson chi-square NB GRD:", nb_grd.pearson_chi2)
print("AIC NB GRD:", nb_grd.aic)
print("Grados de libertad residuales NB GRD:", nb_grd.df_resid)
```

Deviance NB GRD: 6.904915725895279
 Pearson chi-square NB GRD: 4.593740704526928
 AIC NB GRD: 704.1464661059235
 Grados de libertad residuales NB GRD: 29

In []: Para GRD, el modelo Poisson presentó una estadística de dispersión de 2789.36, lo que indica sobredispersión severa. Por esta razón se ajustó usando las mismas covariables y el mismo offset de población. En este modelo estadística clara de asociación con el conteo GRD. Aunque `mean\_schooling\_ch` IRR levemente mayor a 1, sus intervalos de confianza incluyen 1 y sus p-val ENO, no se observa un predictor comunal claramente asociado a la tasa GRD ba

```
In [93]: # PARTE 3.1: SELECCIÓN DEL OUTCOME CONTINUO

# Elegimos grd_mean_los como outcome continuo
# Esta variable representa la duración promedio de estadía hospitalaria en c

outcome_continuo = "grd_mean_los"

# Revisamos la variable seleccionada junto con las covariables del modelo
model_ols_df = analytical[
    ["nombre_comuna", outcome_continuo] + covariates
].dropna().copy()

# Mostramos cuántas observaciones quedan disponibles
print("Outcome continuo seleccionado:", outcome_continuo)
```

```
print("Filas disponibles para modelo OLS:", len(model_ols_df))

# Mostramos las primeras filas
display(model_ols_df.head())
```

Outcome continuo seleccionado: grd_mean_los

Filas disponibles para modelo OLS: 34

	nombre_comuna	grd_mean_los	pct_foreign	mean_schooling_chilean	pct_unemploy
0	Peñalolén	5.34	7.789997	12.222864	34.2795
1	Peñaflor	6.14	3.446953	11.981959	39.8896
2	LAS CONDES	5.92	10.613208	15.967253	29.7105
3	CURACAVÍ	6.49	4.174172	11.494170	39.2087
4	La Reina	5.38	19.380020	12.910000	30.2906

In [94]: # PARTE 3.2: REGRESIÓN LINEAL OLS

```
# Construimos la fórmula del modelo OLS
# La variable dependiente será grd_mean_los
# Las variables explicativas serán las mismas usadas en los modelos de conteo
formula_ols = outcome_continuo + " ~ " + " + ".join(covariates)

# Ajustamos el modelo OLS
ols_grd = smf.ols(
    formula=formula_ols,
    data=model_ols_df
).fit()

# Mostramos el resumen completo del modelo
print(ols_grd.summary())
```

OLS Regression Results					
=====					
==					
Dep. Variable:	grd_mean_los	R-squared:	0.2		
83					
Model:	OLS	Adj. R-squared:	0.1		
84					
Method:	Least Squares	F-statistic:	2.8		
65					
Date:	Sun, 03 May 2026	Prob (F-statistic):	0.04		
09					
Time:	22:12:02	Log-Likelihood:	-28.4		
02					
No. Observations:	34	AIC:	66.		
80					
Df Residuals:	29	BIC:	74.		
44					
Df Model:	4				
Covariance Type:	nonrobust				
=====					
=====					
	coef	std err	t	P> t	[0.0
25	0.975]				

Intercept	5.9873	1.317	4.547	0.000	3.2
94	8.680				
pct_foreign	0.0049	0.013	0.388	0.701	-0.0
21	0.031				
mean_schooling_chilean	0.0773	0.083	0.936	0.357	-0.0
92	0.246				
pct_unemployed	-0.0083	0.009	-0.925	0.363	-0.0
27	0.010				
dependency_ratio	-0.0193	0.008	-2.413	0.022	-0.0
36	-0.003				
=====					
==					
Omnibus:	2.536	Durbin-Watson:	1.8		
38					
Prob(Omnibus):	0.281	Jarque-Bera (JB):	1.6		
02					
Skew:	0.518	Prob(JB):	0.4		
49					
Kurtosis:	3.236	Cond. No.	74		
4.					
=====					
==					

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [95]: # Extraemos una tabla ordenada con coeficientes, errores estándar y p-valores

tabla_ols = pd.DataFrame({
    "coef": ols_grd.params,
```

```

    "std_error": ols_grd.bse,
    "p_value": ols_grd.pvalues,
    "CI_2.5%": ols_grd.conf_int()[0],
    "CI_97.5%": ols_grd.conf_int()[1]
})

display(tabla_ols)

# Reportamos R cuadrado y R cuadrado ajustado
print("R²:", ols_grd.rsquared)
print("R² ajustado:", ols_grd.rsquared_adj)

```

	coef	std_error	p_value	CI_2.5%	CI_97.5%
Intercept	5.987345	1.316776	0.000089	3.294235	8.680455
pct_foreign	0.004940	0.012719	0.700547	-0.021073	0.030953
mean_schooling_chilean	0.077284	0.082557	0.356941	-0.091565	0.246132
pct_unemployed	-0.008259	0.008930	0.362623	-0.026522	0.010004
dependency_ratio	-0.019284	0.007991	0.022358	-0.035627	-0.002941

R²: 0.28322307452507334

R² ajustado: 0.1843572917009455

In []: Se ajustó un modelo OLS usando `grd\_mean\_los` como outcome continuo, ya que la duración promedio de estadía hospitalaria. El modelo obtiene un R² de 0.28 que explica una parte moderada de la variación comunal en la duración promedio. `dependency\_ratio` es la única que presenta evidencia estadística clara, con mayor razón de dependencia tienden a presentar una menor duración promedio. Las otras covariables no presentan evidencia estadística suficiente en esta

In [96]: # PARTE 3.3: DIAGNÓSTICOS DEL MODELO OLS

```

# Obtenemos los valores ajustados por el modelo
# Estos son los valores que el modelo predice para grd_mean_los
fitted_values = ols_grd.fittedvalues

# Obtenemos los residuos del modelo
# Los residuos son la diferencia entre el valor observado y el valor predicho
residuals = ols_grd.resid

# Gráfico de residuos versus valores ajustados
# Sirve para revisar si los residuos se distribuyen de forma aleatoria
plt.figure(figsize=(8, 5))
plt.scatter(fitted_values, residuals)

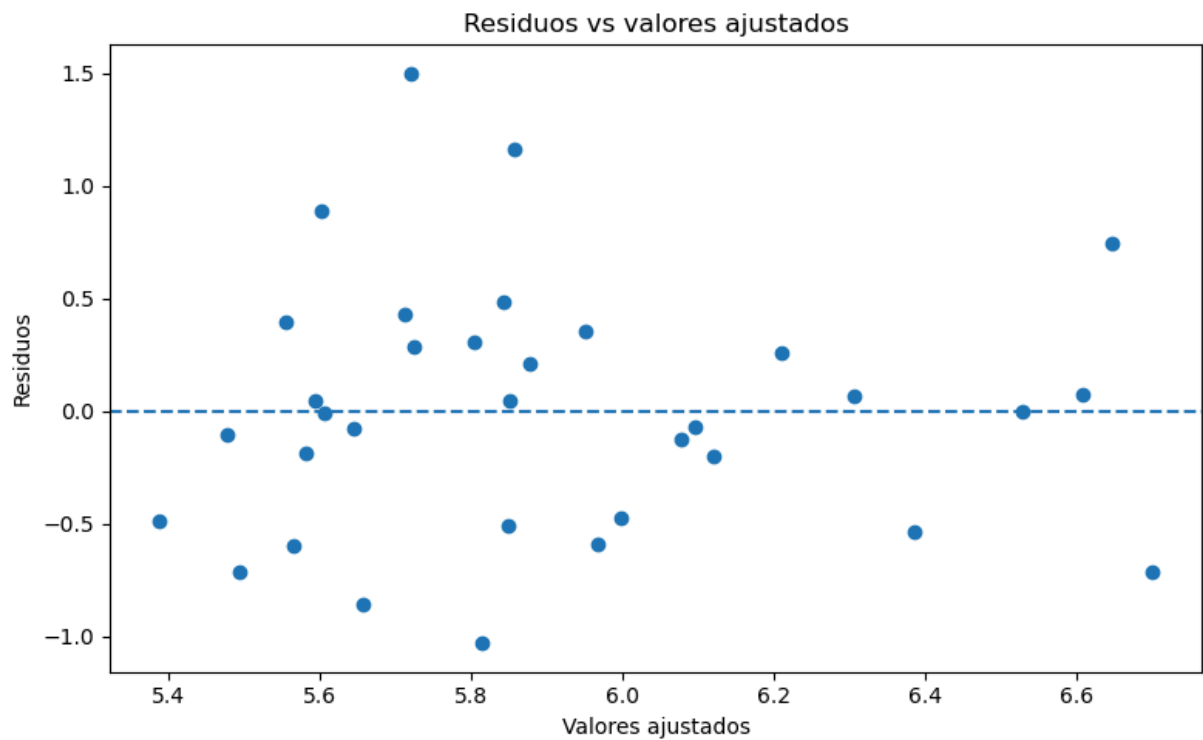
# Agregamos una línea horizontal en cero como referencia
plt.axhline(0, linestyle="--")

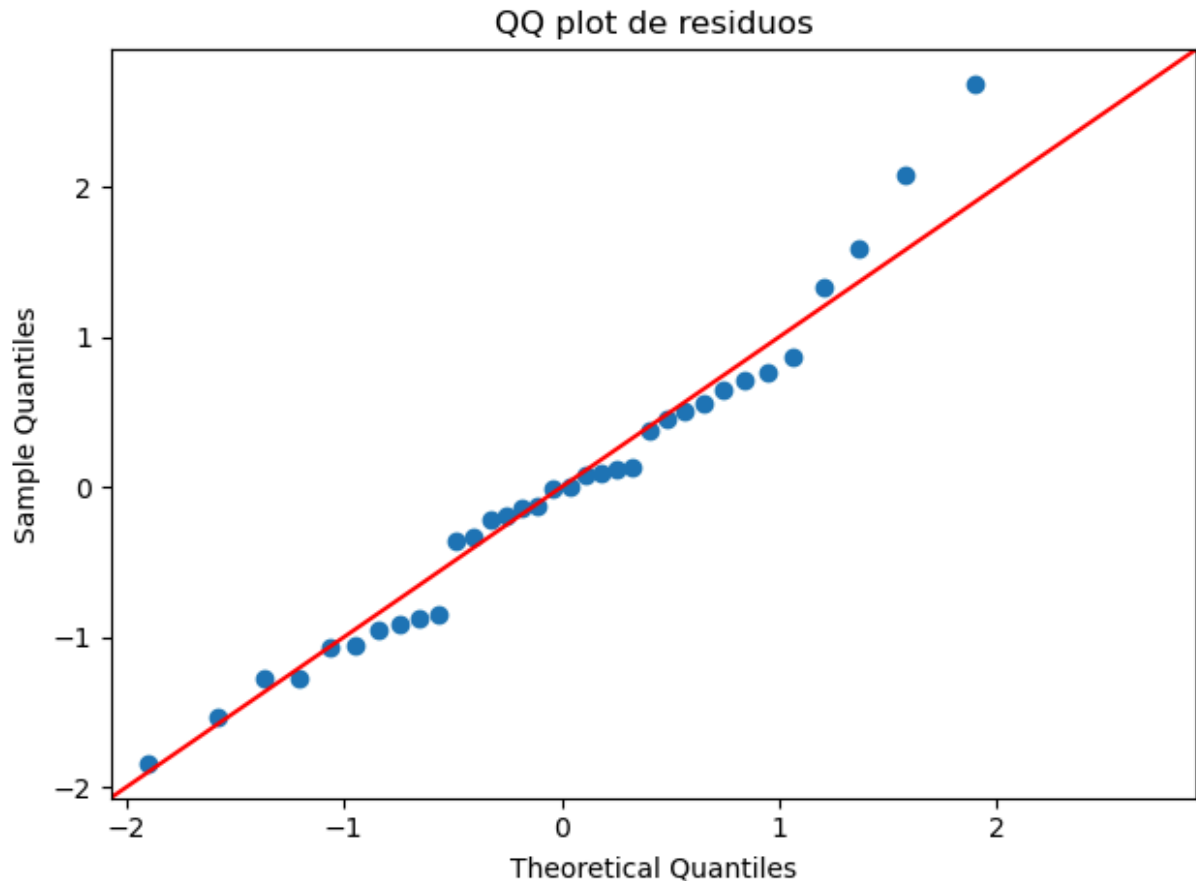
plt.xlabel("Valores ajustados")
plt.ylabel("Residuos")
plt.title("Residuos vs valores ajustados")
plt.tight_layout()

```

```
plt.show()

# QQ plot de residuos
# Sirve para revisar si los residuos se aproximan a una distribución normal
sm.qqplot(residuals, line="45", fit=True)
plt.title("QQ plot de residuos")
plt.tight_layout()
plt.show()
```





In []: El gráfico de residuos versus valores ajustados no muestra un patrón completo. Se observan algunos puntos alejados de la línea cero. En general, los residuos están centrados alrededor de cero, pero con cierta dispersión y posibles observaciones influyentes. Esto sugiere que el modelo puede ser razonable de forma preliminar, aunque no perfecto.

El QQ plot muestra que la mayoría de los residuos siguen relativamente bien la línea central. Sin embargo, en los extremos hay desviaciones, lo que indica que la distribución de los residuos no es perfectamente normal. Como el tamaño muestral es pequeño/moderado y existen comunas con valores extremos, sería probar errores estándar robustos, una transformación del outcome o revisar los supuestos del modelo.

In []: # PARTE 4: FALACIA ECOLÓGICA

El análisis realizado en esta tarea es de tipo ecológico, porque la unidad de análisis es la comuna. Esto significa que cada fila de la base representa un territorio completo y no a individuos. El porcentaje de población extranjera, la escolaridad promedio, la tasa de desempleo, etc., son variables agregadas. La asociación encontrada en los modelos no describe directamente lo que ocurre a nivel individual, sino que describe la asociación entre características promedio o agregadas de comunas.

La diferencia entre una asociación individual y una asociación ecológica es que la primera analiza si una característica de una persona se relaciona con un resultado de salud, mientras que la segunda analiza si una característica de una comuna se relaciona con un resultado de salud. Una asociación ecológica puede ser engañosa porque una persona extranjera tiene mayor probabilidad de presentar una enfermedad crónica, pero esto no necesariamente significa que las personas extranjeras presenten mayores tasas de ENO o GRD. Aunque ambas preguntas parecen similares, la inferencia es diferente. Una relación observada a nivel comunal aplica automáticamente a las personas que viven en esas comunas.

Un ejemplo concreto aparece en el modelo Negative Binomial para ENO. En ese modelo, el coeficiente para la variable extranjera es aproximadamente 1.044, lo que indica que, manteniendo constantes las demás variables, las personas extranjeras tienen un 4.4% más de probabilidad de presentar una enfermedad crónica.

proporción de población extranjera de una comuna se asocia con una tasa esperada sería afirmar que una persona extranjera tiene 4.4% más probabilidad de tener una enfermedad territorial, porque el modelo no observa personas, sino comunas. El resultado territorial, como densidad urbana, acceso al sistema de salud, patrones de concentración de servicios.

Aun así, las asociaciones ecológicas pueden ser útiles para la planificación. El mayor `pct\_foreign` tienden a mostrar mayores tasas ENO, esto puede ayudar a la prevención, traducción de información sanitaria, campañas comunales o fortalecer el resultado no está en culpar o caracterizar a individuos, sino en identificar en detalle las condiciones sociales, sanitarias y administrativas que afectan la salud.

Una amenaza adicional a la inferencia es la presencia de variables omitidas. Los modelos socioeconómicos, pero no consideran factores como densidad poblacional, distancia al centro, cobertura de atención primaria, calidad del registro o diferencias en la autocorrelación espacial, ya que comunas cercanas pueden parecerse entre sí. Los resultados pueden estar afectados por errores de medición o por registros incompletos. Para mejorar los resultados territoriales adicionales, revisar dependencia espacial, usar modelos espaciales si estuvieran disponibles. Esto permitiría distinguir mejor entre asociaciones.

```
In [97]: # PARTE 5.3: COEFFICIENT PLOT DEL MODELO PRINCIPAL

# Usamos la tabla IRR del modelo Negative Binomial de ENO
# Excluimos el intercepto porque no se interpreta como una covariable sustantiva
coef_plot_df = irr_eno_nb.drop(index="Intercept").copy()

# Guardamos los nombres de las variables para el eje Y
coef_plot_df["variable"] = coef_plot_df.index

# Calculamos los errores para graficar los intervalos de confianza
# Estos errores representan la distancia entre el IRR y sus límites inferior y superior
coef_plot_df["error_low"] = coef_plot_df["IRR"] - coef_plot_df["IRR_2.5%"]
coef_plot_df["error_high"] = coef_plot_df["IRR_97.5%"] - coef_plot_df["IRR"]

# Creamos el gráfico
plt.figure(figsize=(8, 5))

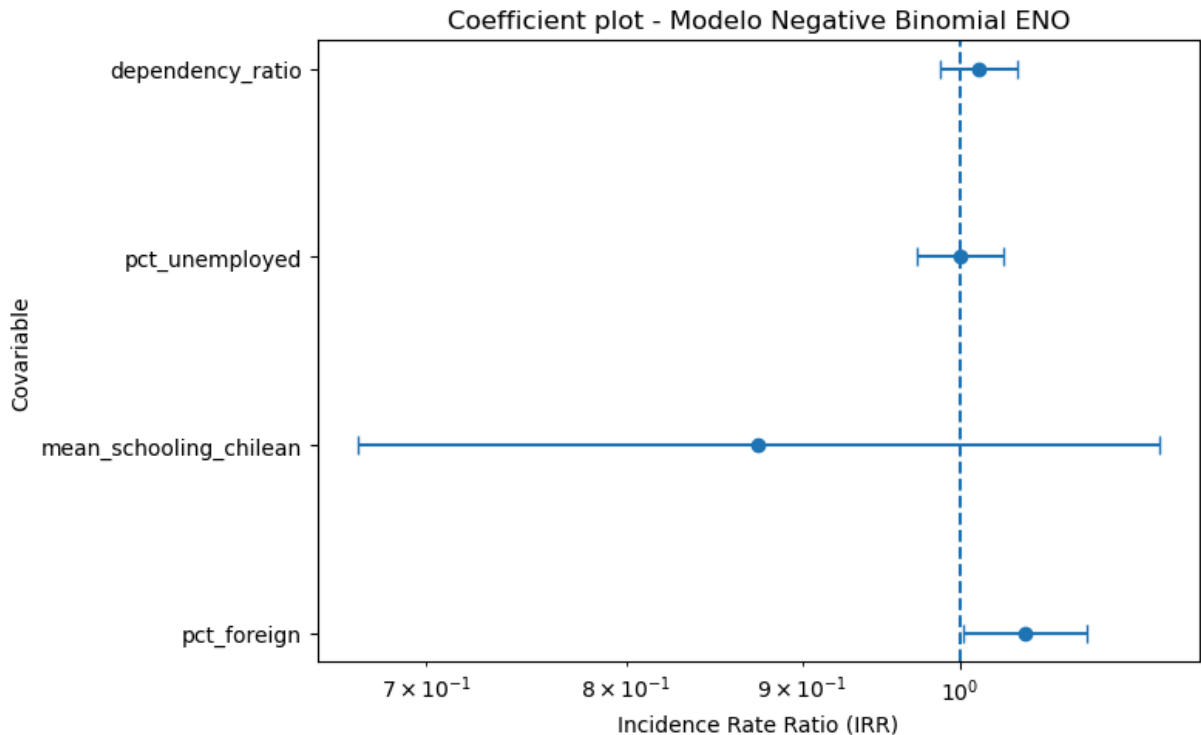
plt.errorbar(
    x=coef_plot_df["IRR"],
    y=coef_plot_df["variable"],
    xerr=[
        coef_plot_df["error_low"],
        coef_plot_df["error_high"]
    ],
    fmt="o",
    capsize=4
)

# Línea vertical en IRR = 1
# Este es el punto nulo: valores sobre 1 aumentan la tasa esperada,
# valores bajo 1 la disminuyen
plt.axvline(1, linestyle="--")

# Usamos escala logarítmica porque los IRR son razones
```

```
plt.xscale("log")

plt.xlabel("Incidence Rate Ratio (IRR)")
plt.ylabel("Covariable")
plt.title("Coefficient plot - Modelo Negative Binomial ENO")
plt.tight_layout()
plt.show()
```



In []: El coefficient plot muestra los IRR del modelo Negative Binomial para ENO y sus intervalos de confianza al 95%. La línea vertical en $IRR = 1$ representa ausencia de asociación. En este modelo, 'pct_foreign' queda por sobre 1 y su intervalo no cruza claramente el valor 1, lo que sugiere una asociación positiva con la tasa esperada de ENO. En cambio, las otras covariables tienen intervalos más amplios que cruzan o se acercan al valor 1, por lo que no muestran evidencia clara de asociación.

PARTE 6: SÍNTESIS INTEGRADA

El análisis integrado de la Región Metropolitana muestra una fuerte heterogeneidad comunal en términos demográficos y sanitarios. Desde la dimensión censal, las comunas presentan diferencias relevantes en tamaño poblacional, porcentaje de población extranjera, escolaridad promedio, empleo y razón de dependencia. Una de las principales dimensiones de variación parece estar asociada a la composición sociodemográfica de las comunas: algunas, como Independencia, Santiago, San Miguel y Recoleta, concentran mayores porcentajes de población extranjera, mientras que otras presentan valores mucho más bajos. También se observan diferencias importantes en escolaridad promedio y dependencia demográfica, lo que sugiere que la Región Metropolitana no puede entenderse como un territorio homogéneo.

Al cruzar esta información con los resultados de salud de ENO y GRD, aparecen patrones que no son completamente lineales. En la matriz de correlación, `pct_foreign` muestra una asociación positiva moderada con `eno_rate_per_10k`, pero una relación mucho más débil con `grd_rate_per_10k`. Esto indica que las comunas con mayor proporción de población extranjera tienden a mostrar mayores tasas ENO, aunque esta relación no se replica con la misma claridad en GRD. Por otro lado, variables como `mean_schooling_chilean` y `mean_schooling_foreign` están fuertemente correlacionadas entre sí, lo que muestra que la escolaridad promedio de distintos grupos poblacionales tiende a moverse en conjunto a nivel comunal. Sin embargo, su relación con los outcomes de salud no es necesariamente fuerte ni directa.

Los gráficos bivariados permiten observar comunas que se alejan de las tendencias generales. En ENO, San Pedro aparece como un outlier relevante, con una tasa por 10.000 habitantes mucho más alta que la mayoría de las comunas. También destacan Santiago y Valparaíso, aunque Santiago fue revisada cuidadosamente porque inicialmente tenía un problema de escala en `pop_total`. En GRD, Peñalolén aparece con una tasa elevada, mientras que Tiltil muestra una tasa considerablemente baja. Estos casos muestran que los promedios o correlaciones generales pueden ocultar comportamientos comunales específicos que requieren análisis contextual.

Los modelos de conteo agregan información importante más allá de la matriz de correlación. En el modelo Poisson para ENO, varias covariables aparecían como estadísticamente significativas, pero el diagnóstico mostró sobredispersión severa, con una estadística de dispersión de 361.23. Por eso se ajustó un modelo Negative Binomial, más apropiado para datos de conteo con varianza mayor que la media. En este modelo, `pct_foreign` se mantuvo como la covariable más relevante, con un IRR aproximado de 1.044. Esto sugiere que, manteniendo constantes las demás variables, un aumento de 1 punto porcentual en población extranjera se asocia con una tasa esperada de ENO cerca de 4.4% mayor. En cambio, `mean_schooling_chilean`, `pct_unemployed` y `dependency_ratio` dejaron de mostrar evidencia estadística clara.

Para GRD, el patrón fue distinto. El modelo Poisson también presentó sobredispersión severa, con una estadística de dispersión de 2789.36, por lo que se usó Negative Binomial. En este caso, ninguna covariable mostró una asociación estadísticamente clara con el conteo GRD. Esto sugiere que los factores que explican la tasa GRD pueden ser distintos a los incluidos en el modelo, o que se requiere incorporar información adicional sobre oferta hospitalaria, derivaciones, estructura etaria, acceso a servicios o severidad de casos.

Finalmente, el modelo OLS para `grd_mean_los` mostró un R^2 de 0.283 y un R^2 ajustado de 0.184. La única variable con evidencia estadística clara fue `dependency_ratio`, con un coeficiente negativo. Esto sugiere una asociación entre composición demográfica y duración promedio de estadía, aunque el ajuste general del modelo es moderado. En conjunto, la Tarea 3 muestra que integrar Census, ENO y GRD

permite observar relaciones que no serían visibles usando una sola fuente. Sin embargo, todos los resultados deben interpretarse como asociaciones ecológicas a nivel comunal y no como relaciones causales individuales.